

Teaching Exam

Subject – Computer Science

Topic – Database Query using SQL



Lecture No.- 2

By- Rahul Sir

Topics to be Covered

Topic



- 1:- LOGICAL OPERATOR IN MYSQL ✓
- 2:- DISTINCT KEYWORD ✓
- 3:- CREATE CONSTRAINT ✓
- 4:- AGGREGATE FUNCTION ✓
- 5:- LIKE, BETWEEN, IN OPERATOR ✓
- 6:- JOINS ✓
- 7:- ORDER BY ✓
- 8:- GROUP BY ✓
- 9:- HAVING CLAUSE ✓
- 10:- CORRECT SEQUENCE OF SQL STATEMENT ✓
- 11:- PRACTICE ON GROUP BY AND HAVING CLAUSE ✓



Topic : SQL



TEACHING
WALLAH

SQL, which means “**Structured Query Language**,” is the primary language used for managing and manipulating relational databases. We use SQL commands to tell the database what actions we want it to perform.

What are SQL commands?

SQL commands are the keywords that tell the database what kind of action to perform, like CREATE, SELECT, or INSERT. These commands perform various database operations, such as creating tables, inserting data, querying information, and controlling access and security.

SQL Statement: It is a complete instruction you give to the database. It is made up of one or more commands and other elements like table names, columns, values, or conditions. For example, “CREATE TABLE Customers” is a statement.

A **SQL Query** is a type of SQL statement that asks the database to return data. It always starts with the SELECT command.



Topic : Types of SQL Query

SQL commands are divided into five main groups based on their use. Knowing these groups helps you understand what different commands are meant to do ,There five types of SQL command:-

Type	Full Form	What it does	Example
DDL ✓	Data Definition Language	Makes or changes tables	CREATE, ALTER, DROP ✓
DML ✓	Data Manipulation Language	Adds, changes, and deletes data	INSERT, UPDATE, DELETE ✓
DQL ✓	Data Query Language	Asks questions to get data	SELECT ✓
DCL ✓	Data Control Language	Controls who can access data	GRANT, REVOKE ✓
TCL ✓	Transaction Control Language	Manages changes as one group	COMMIT, ROLLBACK ✓



Topic : Sub-Query

What is SQL Subquery?

In SQL, a subquery can be defined as a query **embedded** within **another query**. It is often used in the **WHERE**, **HAVING**, or **FROM** clauses of a statement. Subqueries are commonly used with **SELECT**, **UPDATE**, **INSERT**, and **DELETE** statements to achieve **complex filtering** and **data manipulation**. They are an essential tool when we need to perform operations like:

- **Filtering**: Getting specific records based on conditions derived from another query.
- **Aggregating**: Performing aggregate functions like SUM, COUNT, or AVG based on subquery results.
- **Updating**: Dynamically updating records based on values from other tables.
- **Deleting**: Deleting records from one table using criteria based on another.

While there is no universal syntax for subqueries, they are commonly used in **SELECT** statements as follows. This general syntax allows the outer query to use the results of the inner subquery for filtering or other operations.



Topic : Logical Operators in MySQL

The logical operators in MySQL are fundamental components of SQL statements that allow you to filter data from a table by combining numerous conditions in a WHERE clause. Logical operators are used in the creation of complicated queries that retrieve data based on specified parameters. MySQL has three logical operators: AND, OR, and NOT.

Truth Tables for OR, AND, and NOT Operators

X	Y	<u>X OR Y</u>	<u>X AND Y</u>	<u>NOT X</u>
true	true	true	true	false
true	false	true	false	false
false	true	true	false	true
false	false	false	false	true

$$2^1 = 2$$

(A)

$$Y = \bar{A} = \text{NOT A}$$

True
False

False
True

Q

(A)

(B)

$$Y = A \& B$$

A AND B

True
True
False

True
false
True

T
F
F

False

True

F

False

False

F



Topic : Logical AND Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

SELECT * FROM employees
WHERE department = 'Sales'
AND salary > 50000;

Select * from employees

where department = 'Sales';

Output:

name	age	department	salary
B	29	Sales	60000

OR

B 29 Sales 60000
C 35 Sales 45000
D 26 IT 52000



Topic : Logical OR Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

```
SELECT * FROM employees  
WHERE department = 'Sales'  
OR department =  
'Marketing';
```

Output:

name	age	department	salary
A	25	Marketing	60000
B	29	Sales	60000



Topic : Logical NOT Operators in MySQL

name	age	department	salary
A	25	IT	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

```
SELECT * FROM employees  
WHERE NOT department =  
'Sales';
```

department != 'Sales'

< >

Output:

name	age	department	salary
A	25	Marketing	60000
D	26	IT	52000



Topic :Explanation of Operator Precedence in MySQL

The order in which logical operators in MySQL are evaluated in a SQL query is called logical operator precedence in MySQL. **The precedence of logical operators is: NOT > AND > OR.** The higher precedence operators are evaluated first, followed by lower precedence operators. Let us understand how operator precedence works in MySQL with the help of our sample employees table.

name	age	department	salary
A	25	Marketing	60000
B	29	Sales	60000
C	35	Sales	45000
D	26	IT	52000

Handwritten notes above the table:
& || !
AND OR NOT
A bracket is drawn around row A.
A bracket is drawn around row B, with the word "Sales" written next to it.

SELECT * FROM employees
WHERE department = 'Sales' AND salary > 50000 OR department = 'Marketing';

Handwritten notes:
A bracket is drawn around the first condition.
A bracket is drawn around the second condition.
A bracket is drawn around the OR operator.
Handwritten text: AND NOT AND > OR

Output:

name	age	department	salary
✓ B	29	Sales	60000
✓ A	25	Marketing	60000

Handwritten notes:
A bracket is drawn around the first two rows.
Handwritten text: OR



What is the SQL DISTINCT Keyword?

The distinct keyword is used in conjunction with the select keyword. It is helpful when there is a need to avoid duplicate values present in any specific columns/table. When we use distinct keywords only the **unique values** are fetched.

SYNTAX:-

```
SELECT DISTINCT column1, column2  
FROM table_name;
```

Select Distinct City from x42;

City
Agra
Mathura
Delhi
Agra

Agra
Mathura
Delhi

NOTE: If a distinct keyword is used with multiple columns, the distinct combination is displayed in the result set.



Topic :SQL DISTINCT Keyword

```
CREATE TABLE students (  
    ROLL_NO INT,  
    NAME VARCHAR(50),  
    ADDRESS VARCHAR(100),  
    PHONE VARCHAR(20),  
    AGE INT  
);  
  
INSERT INTO students (ROLL_NO, NAME, ADDRESS, PHONE, AGE)  
VALUES  
    (1, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23),  
    (2, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23),  
    (3, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26),  
    (4, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22),  
    (5, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27),  
    (6, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24),  
    (7, 'Shubham Kumar', '123 Main Street, Bangalore', '9876543210', 23), --  
Duplicate  
    (8, 'Shreya Gupta', '456 Park Road, Mumbai', '9876543211', 23), --  
Duplicate  
    (9, 'Naveen Singh', '789 Market Lane, Delhi', '9876543212', 26), --  
Duplicate  
    (10, 'Aditya Patel', '7898 Ocean Drive, Chennai', '9876543214', 27), --  
Duplicate  
    (11, 'Aman Chopra', '246 Forest Avenue, Kolkata', '9876543213', 22), --  
Duplicate  
    (12, 'Avdeep Desai', '34 River View, Hyderabad', '9876543215', 24); --  
Duplicate
```




Topic :SQL DISTINCT Keyword

Output:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Shubham Kumar	123 Main Street, Bangalore	9876543210	23
2	Shreya Gupta	456 Park Road, Mumbai	9876543211	23
3	Naveen Singh	789 Market Lane, Delhi	9876543212	26
4	Aman Chopra	246 Forest Avenue, Kolkata	9876543213	22
5	Aditya Patel	7898 Ocean Drive, Chennai	9876543214	27
6	Avdeep Desai	34 River View, Hyderabad	9876543215	24
7	Shubham Kumar	123 Main Street, Bangalore	9876543210	23
8	Shreya Gupta	456 Park Road, Mumbai	9876543211	23
9	Naveen Singh	789 Market Lane, Delhi	9876543212	26
10	Aditya Patel	7898 Ocean Drive, Chennai	9876543214	27
11	Aman Chopra	246 Forest Avenue, Kolkata	9876543213	22
12	Avdeep Desai	34 River View, Hyderabad	9876543215	24



Topic :SQL DISTINCT Keyword

Example 1: Fetch Unique Names from the NAME Field.

Query:

```
SELECT DISTINCT NAME FROM students;
```

Output:

NAME	
Shubham Kumar	✓
Shreya Gupta	✓
Naveen Singh	✓
Aman Chopra	✓
Aditya Patel	✓
Avdeep Desai	✓

output

The query returns only unique names, eliminating the duplicate entries from the table.



Topic :SQL DISTINCT Keyword

Example 2: Fetching Unique Combinations of Multiple Columns

If you want to retrieve unique combinations of NAME and AGE

Query:

```
SELECT DISTINCT NAME, AGE FROM students;
```

Output:

NAME	AGE
Shubham Kumar	23
Shreya Gupta	23
Naveen Singh	26
Aman Chopra	22
Aditya Patel	27
Avdeep Desai	24

23
23
23
Select *



Topic : Create Constraints

Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement.

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    Age int Check Age >= 15  
);
```

Address column2 datatype constraint Not Null

Roll int ^{Primary Key} ~~NOT NULL~~
Name char
Mobino int unique,

The following constraints are commonly used in SQL:

- NOT NULL - Ensures that a column cannot have a NULL value
- UNIQUE - Ensures that all values in a column are different
- PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- FOREIGN KEY - Prevents actions that would destroy links between tables
- CHECK - Ensures that the values in a column satisfies a specific condition
- DEFAULT - Sets a default value for a column if no value is specified



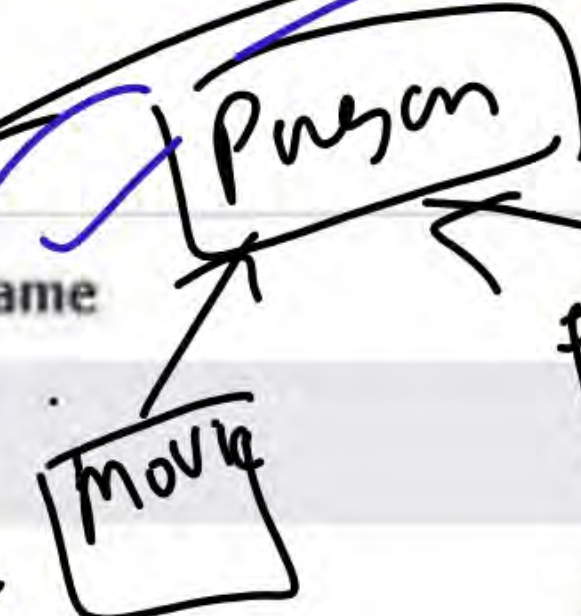
Topic : Create Constraints – Foreign Key

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Gia	30
2	Svendson	Tora	23
3	Batterman	Karl	20

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Movie Name	Timing	PersonID



Order create Table Persons(
PersonID int PrimaryKey,
FirstName char(20),
LastName char(20),
Age int);

create Table Movie(
Movie Name varchar,
Timing Time,
PersonID int);

```
CREATE TABLE Orders (  
OrderID int NOT NULL,  
OrderNumber int NOT NULL,  
PersonID int,  
PRIMARY KEY (OrderID),  
FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

Primary Key (OrderID),
foreignKey (PersonID) Reference Persons(PersonID)
int task name
PK of parent table



Topic : Create Constraints – Default

The following SQL sets a **DEFAULT** value for the "City" column when the "Persons" table is created:

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName nam varchar(255) NOT NULL,  
    FirstName nam varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Sandnes'  
);
```

IP	nam	Age	City
1	X	10	Sadms
2	Y	12	Sadms
3	Z	14	Sadms



Topic : Create Constraints – Default

The **DEFAULT** constraint can also be used to insert system values, by using functions like **CURRENT_DATE()**:

```
CREATE TABLE Orders (  
  ID int NOT NULL,  
  OrderNumber int NOT NULL,  
  OrderDate date DEFAULT CURRENT_DATE()  
);
```

(current Date function())
function()
Datatype
function

DEFAULT on ALTER TABLE

To create a **DEFAULT** constraint on the "City" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons  
ALTER City SET DEFAULT 'Sandnes';
```




Topic : Create Constraints – Check

The following SQL creates a **CHECK** constraint on the "Age" column when the "Persons" table is created. The **CHECK** constraint ensures that the age of a person must be 18, or older:

```
CREATE TABLE Persons (  
  ✓ ID int NOT NULL,  
  ✓ LastName varchar(255) NOT NULL,  
  ✓ FirstName varchar(255),  
  ✓ Age int,  
    CHECK (Age >= 18) ✓  
);
```




Topic : Create Constraints – Check

To allow naming of a **CHECK** constraint, and for defining a **CHECK** constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255),  
    CONSTRAINT CHK_Person CHECK (Age >= 18 AND City = 'Sandnes')  
);
```

Constraint Name

Constraint CK2 (check (Email NOT Null),



Topic :SQL Aggregate Functions



TEACHING
WALLAH

An aggregate function is a function that performs a calculation on a set of values, and returns a single value.

Aggregate functions are often used with the **GROUP BY** clause of the **SELECT** statement. The **GROUP BY** clause splits the result-set into groups of values and the aggregate function can be used to return a single value for each group.

The most commonly used SQL aggregate functions are:

- MIN() - returns the smallest value within the selected column
- MAX() - returns the largest value within the selected column
- COUNT() - returns the number of rows in a set
- SUM() - returns the total sum of a numerical column
- AVG() - returns the average value of a numerical column

Aggregate functions ignore null values (except for **COUNT()**).



Topic :SQL Aggregate Functions- MIN() and MAX()



TEACHING
WALLAH

The **MIN()** function returns the smallest value of the selected column.

The **MAX()** function returns the largest value of the selected column.

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35



Topic :SQL Aggregate Functions- MIN() and MAX()



TEACHING
WALLAH

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35

Set Column Name (Alias)

Smallest price
10

When you use **MIN()** or **MAX()**, the returned column will not have a descriptive name. To give the column a descriptive name, use the **AS** keyword:

Example

```
SELECT MIN(Price) AS SmallestPrice  
FROM Products;
```

Select ~~MIN~~^{Max}(Marks) from class2;



Topic :SQL Aggregate Functions- SUM(), AVG()

Give the summarized column a name by using the **AS** keyword.

Example

Name the column "total":

```
SELECT SUM(Quantity) AS total  
FROM OrderDetails;
```

Select Sum(Quantity)

The SQL AVG() Function

The **AVG()** function returns the average value of a numeric column.

Example

Find the average price of all products:

```
SELECT AVG(Price) AS AveragePrice  
FROM Products;
```

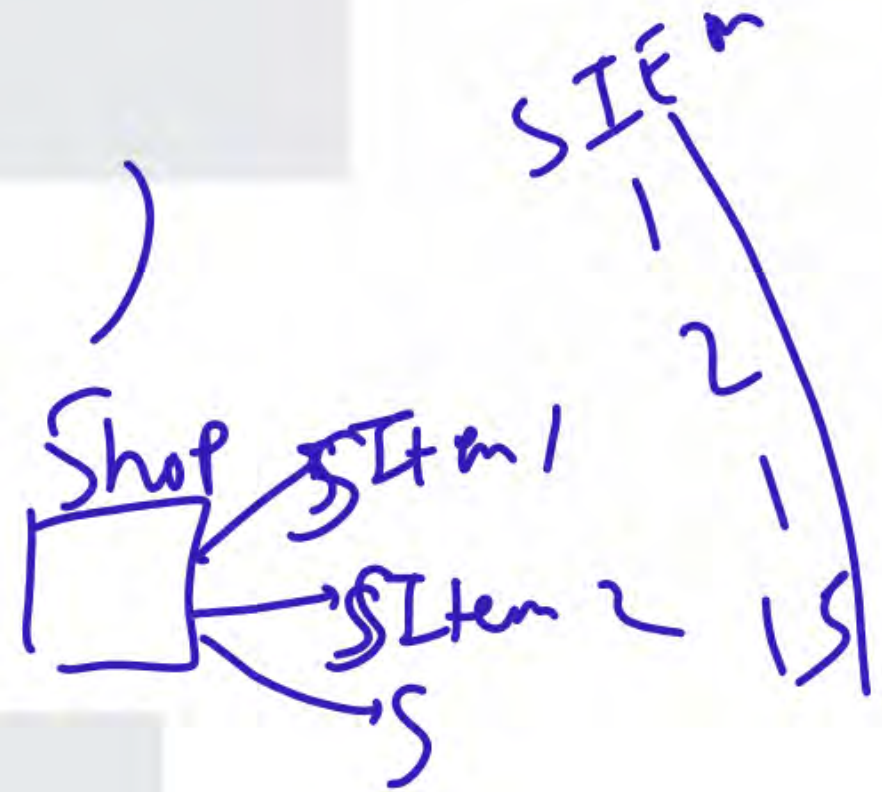
400
200
350

$$\frac{950}{3}$$

85%

90%

23





The SQL LIKE Operator

The **LIKE** operator is used in a **WHERE** clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the **LIKE** operator:

- The percent sign **%** represents zero, one, or multiple characters
- The underscore sign **_** represents one, single character

Example

Select all customers that starts with the letter "a":

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```

Handwritten notes and examples:

- % Gupta** (with a circled % and an arrow pointing to it)
- UPI 1234** (with an arrow pointing to the % sign in the original image)
- UTR 4127**
- UPI 4925**
- AVI** (with an arrow pointing to the % sign in the original image)
- ABC**
- ABhishek** (with an arrow pointing to the % sign in the original image)
- Aravind** (with an arrow pointing to the % sign in the original image)
- Any Multiple Data** (with an arrow pointing to the % sign)
- field value** (with an arrow pointing to the % sign)



The SQL IN Operator

The **IN** operator allows you to specify multiple values in a **WHERE** clause.

The **IN** operator is a shorthand for multiple **OR** conditions.

Example

Return all customers from 'Germany', 'France', or 'UK'

```
SELECT * FROM Customers  
WHERE Country IN ( 'Germany', 'France', 'UK' );
```




The SQL BETWEEN Operator

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

The **BETWEEN** operator is inclusive: begin and end values are included.

Example

Selects all products with a price between 10 and 20:

```
SELECT * FROM Products  
WHERE Price BETWEEN 10 AND 20;
```




Topic :SQL INNER JOIN



TEACHING
WALLAH

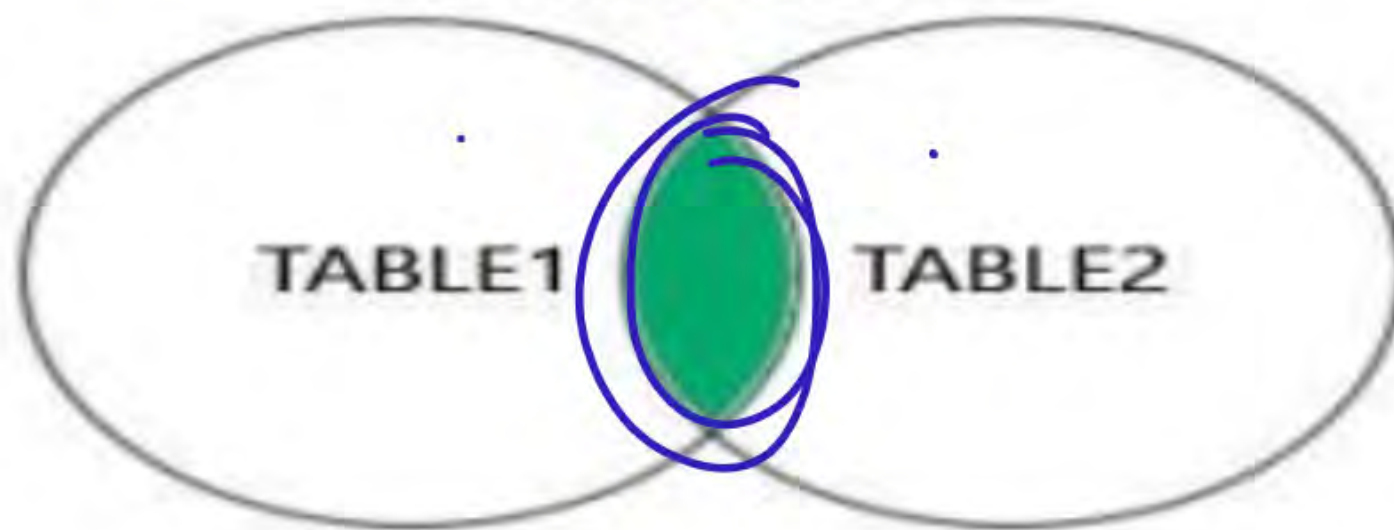
Join Products and Categories with the INNER JOIN keyword:

```
SELECT ProductID, ProductName, CategoryName  
FROM Products  
INNER JOIN Categories ON Products.CategoryID = Categories.CategoryID
```

first Table
Second Table name



INNER JOIN





Topic :SQL LEFT OUTER JOIN



TEACHING
WALLAH

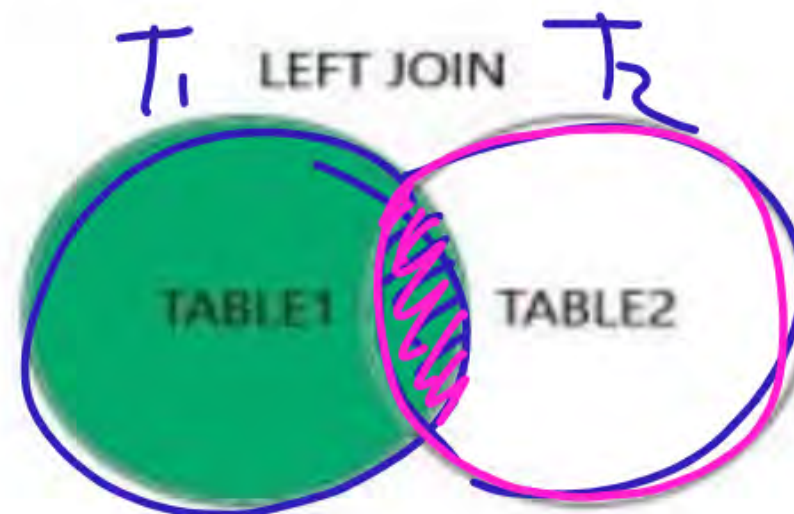
The **LEFT JOIN** keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

LEFT JOIN Syntax

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Handwritten notes: PK is written above table1.column_name and FK is written above table2.column_name.

Note: In some databases LEFT JOIN is called LEFT OUTER JOIN.





Topic :SQL RIGHT OUTER JOIN



TEACHING
WALLAH

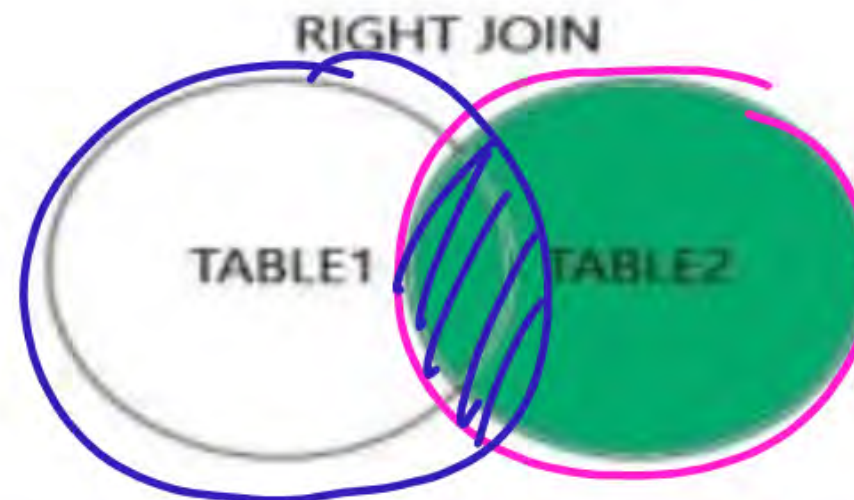
The **RIGHT JOIN** keyword returns all records from the right table (table2), and the matching records from the left table (table1). The result is 0 records from the left side, if there is no match.

RIGHT JOIN Syntax

```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

JN $T_1 \cdot PK = T_2 \cdot FK$

Note: In some databases **RIGHT JOIN** is called **RIGHT OUTER JOIN**.





Topic :SQL FULL OUTER JOIN



TEACHING
WALLAH

The **FULL OUTER JOIN** keyword returns all records when there is a match in left (table1) or right (table2) table records.

Tip: **FULL OUTER JOIN** and **FULL JOIN** are the same.

FULL OUTER JOIN Syntax

```
SELECT column_name(s)  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column_name = table2.column_name  
WHERE condition;
```





Topic :ORDER BY

DESC

The **ORDER BY** keyword sorts the records in ascending order by default. To sort the records in descending order, use the **DESC** keyword.

Example

Sort the products from highest to lowest price:

```
SELECT * FROM Products  
ORDER BY Price DESC;
```

DESC;

↓ column name

price

Using Both ASC and DESC

The following SQL statement selects all customers from the "Customers" table, sorted ascending by the "Country" and descending by the "CustomerName" column:

Example

```
SELECT * FROM Customers  
ORDER BY Country ASC, CustomerName DESC;
```

Country ASC, CustomerName DESC



GROUP BY Clause with Aggregate Functions

Typically, we group the record of a table to perform calculations on them. Therefore, the SQL GROUP BY clause is often used with the aggregate functions such as SUM(), AVG(), MIN(), MAX(), COUNT(), etc.

Example

Assume we have created a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary, using the following query –



Topic :GROUP BY



TEACHING
WALLAH

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh //	32 ✓	Ahmedabad	2000.00
2	Khilan //	25 //	Delhi ✓	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali /	25 /	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

8 Shyam 32 Mirzapur 50,000

COMMAND:- ^{→ No of Count}
SELECT AGE, COUNT(Name) FROM CUSTOMERS GROUP BY AGE;

Ahm 25000 + 27000
32
Kb
23
Delhi 20000
25
Delhi 30000
25
50,000



COMMAND:-

SELECT **AGE**, COUNT(Name) FROM CUSTOMERS GROUP BY **AGE**;

Output

Following is the result produced –

AGE	COUNT(Name)
32	1
25	2
23	1
27	1
22	1
24	1



Topic :GROUP BY



TEACHING
WALLAH

COMMAND:-

```
SELECT ADDRESS, AGE, SUM(SALARY) AS TOTAL_SALARY FROM  
CUSTOMERS GROUP BY ADDRESS, AGE;
```

Output

This would produce the following result –

select Name, Max(score) from JSE Groupby Name;

ADDRESS	AGE	TOTAL_SALARY
Ahmedabad	32	2000.00
Delhi	25	1500.00
Kota	23	2000.00
Mumbai	25	6500.00
Bhopal	27	8500.00
Hyderabad	22	4500.00
Indore	24	10000.00



Topic :GROUP BY With Having Clause

The table created is as shown below –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

SELECT ADDRESS, AGE, MIN(SALARY) AS MIN_SUM FROM CUSTOMERS GROUP BY ADDRESS,
AGE HAVING AGE > 25;



Topic :GROUP BY With Having Clause



TEACHING
WALLAH

SELECT ADDRESS, AGE, MIN(SALARY) AS MIN_SUM FROM CUSTOMERS GROUP BY ADDRESS, AGE HAVING AGE > 25;

Output

The result produced is as follows –

ADDRESS	AGE		MIN_SUM
Ahmedabad	32	/	2000.00
Bhopal	27	/	8500.00



SELECT > FROM > WHERE > GROUP BY >
HAVING BY > ORDER BY



Topic : HAVING with ORDER BY Clause

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali Rahul	25 25	Mumbai Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

4, 5, 7

$$\begin{array}{r} 6500 \\ 7000 \\ \hline = 13500 \end{array}$$

SELECT ADDRESS, AGE, SUM(SALARY) AS TOTAL_SALARY FROM CUSTOMERS GROUP
BY ADDRESS, AGE HAVING TOTAL_SALARY >= 5000 ORDER BY TOTAL_SALARY DESC;



Topic : HAVING with ORDER BY Clause

✓ ✓ ✓
SELECT ADDRESS, AGE, SUM(SALARY) AS TOTAL_SALARY FROM CUSTOMERS GROUP BY ADDRESS, AGE HAVING TOTAL_SALARY >=5000 ORDER BY TOTAL_SALARY DESC;

Output

The result produced is as follows –

ADDRESS	AGE	TOTAL_SALARY
Indore	24	10000.00
Bhopal	27	8500.00
<u>Mumbai</u>	<u>25</u>	6500.00 13500



THANK YOU